

Aplicación del Patrón de Diseño Strategy en el Proyecto MathemaX

Daniel Pérez Amundaray, Manuel Antonio Sánchez Venegas,

Evan Marín Jiménez y Sebastián Jiménez Rojas

Escuela de Ingeniería en Sistemas de Computación, Universidad Fidélitas

SC-601: Programación Avanzada

Prof. Álvaro Andrei Miranda Muñoz

20 de julio de 2026

Aplicación del Patrón de Diseño Strategy en el Proyecto MathemaX

Patrón Seleccionado

Para el proyecto MathemaX, aplicación web educativa en ASP.NET MVC con C#, se seleccionó el patrón de comportamiento **Strategy** (Gamma et al., 1994; Refactoring.Guru, s.f.-b).

En Qué Consiste el Patrón

Strategy permite definir una familia de algoritmos, colocar cada uno en una clase separada y hacer sus objetos intercambiables (Refactoring.Guru, s.f.-b). Una clase llamada *contexto* delega el trabajo en un objeto que implementa una interfaz común, sin conocer la clase concreta; así pueden agregarse nuevas variantes del algoritmo sin modificar el código existente.

Por Qué se Eligió Este Patrón

El proyecto define tres modos de juego en la enumeración ModoJuego (Operador Perdido, Contrarreloj y Secuencias Lógicas), pero la lógica de resolver retos existe solo en OperadorPerdidoBusiness. Replicar esa clase por modo duplicaría el flujo común (obtener reto, validar, calcular XP y registrar la partida), y un switch por modo violaría el principio abierto/cerrado. Con Strategy, cada modo se vuelve una clase intercambiable dentro de la capa AP.Core, sin afectar controladores ni repositorios, y cada estrategia puede probarse aislada en AP.Tests.

Implementación en el Proyecto

Se define la interfaz en AP.Core/Business/Estrategias y una estrategia concreta por modo de juego:

```
public interface IModoJuegoStrategy
{
    ModoJuego Modo { get; }
    bool ValidarRespuesta(Reto reto, string respuestaUsuario);
    int CalcularXp(Reto reto, bool acertado, int tiempoEmpleadoSegundos);
}

public class OperadorPerdidoStrategy : IModoJuegoStrategy
{
```

```

    public ModoJuego Modo => ModoJuego.OperadorPerdido;

    public bool ValidarRespuesta(Reto reto, string respuestaUsuario)
    {
        return string.Equals((respuestaUsuario ?? "").Trim(),
            reto.RespuestaCorrecta.Trim(), StringComparison.OrdinalIgnoreCase);
    }

    public int CalcularXp(Reto reto, bool acertado, int tiempoEmpleadoSegundos)
    {
        return acertado ? reto.Dificultad * 10 : 0;
    }
}

public class ContrarrelojStrategy : IModoJuegoStrategy
{
    public ModoJuego Modo => ModoJuego.Contrarreloj;

    public bool ValidarRespuesta(Reto reto, string respuestaUsuario)
    {
        return string.Equals((respuestaUsuario ?? "").Trim(),
            reto.RespuestaCorrecta.Trim(), StringComparison.OrdinalIgnoreCase);
    }

    public int CalcularXp(Reto reto, bool acertado, int tiempoEmpleadoSegundos)
    {
        if (!acertado) return 0;
        int bono = Math.Max(0,
            (reto.TiempoLimiteSegundos - tiempoEmpleadoSegundos) / 5);
        return reto.Dificultad * 10 + bono; // bono por rapidez
    }
}

// SecuenciasLogicasStrategy valida el siguiente numero de la secuencia
// con int.TryParse y otorga reto.Dificultad * 12 de XP.

```

La clase PartidaBusiness actúa como contexto: selecciona la estrategia según el modo del reto y ejecuta el flujo común sin condicionales:

```

public class PartidaBusiness
{
    private readonly IRepositoryReto _repositoryReto;
    private readonly IRepositoryPartida _repositoryPartida;
    private readonly Dictionary<ModoJuego, IModoJuegoStrategy> _estrategias;

    public PartidaBusiness(IRepositoryReto repositoryReto,
        IRepositoryPartida repositoryPartida,
        IEnumerable<IModoJuegoStrategy> estrategias)
    {
        _repositoryReto = repositoryReto;
        _repositoryPartida = repositoryPartida;
        _estrategias = estrategias.ToDictionary(e => e.Modo);
    }

    public ResultadoReto ResolverReto(int retoId, string usuarioId,
        string respuestaUsuario, int tiempoEmpleadoSegundos)
    {
    }
}

```

```

{
    var reto = _repositoryReto.GetById(retoId);
    if (reto == null)
        throw new AppException("El reto solicitado ya no existe.");

    if (!_estrategias.TryGetValue(reto.Modos, out var estrategia))
        throw new AppException("Modo de juego no soportado.");

    var dentroDeTiempo =
        tiempoEmpleadoSegundos <= reto.TiempoLimiteSegundos;
    var acertado = dentroDeTiempo
        && estrategia.ValidarRespuesta(reto, respuestaUsuario);
    var xpGanado = estrategia.CalcularXp(reto, acertado,
        tiempoEmpleadoSegundos);

    _repositoryPartida.Add(new Partida
    {
        UsuarioId = usuarioId, RetoId = retoId, Acertado = acertado,
        TiempoEmpleadoSegundos = tiempoEmpleadoSegundos,
        XpGanado = xpGanado, FechaJuego = DateTime.Now
    });

    return new ResultadoReto
    {
        Acertado = acertado, RespuestaCorrecta = reto.RespuestaCorrecta,
        XpGanado = xpGanado, FueraDeTiempo = !dentroDeTiempo, Reto = reto
    };
}
}

```

Con esta estructura, agregar un cuarto modo de juego solo requiere crear una nueva estrategia y registrarla en el contexto, sin modificar el código existente. Esto elimina duplicación, cumple el principio abierto/cerrado y mantiene la separación de capas del proyecto.

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Refactoring.Guru. (s.f.-a). *Patrones de diseño en C#*.

<https://refactoring.guru/design-patterns/csharp>

Refactoring.Guru. (s.f.-b). *Strategy en C#*.

<https://refactoring.guru/design-patterns/strategy/csharp/example>